

```
@given(u'egg setting is {egg_hardness}')
def impl(context, egg_hardness):
    context.machine.egg_hardness = egg_hardness

@given(u'amount of eggs to boil is set to {amount}')
def impl(context, amount):
    context.machine.eggs_to_boil = int(amount)
```

Up to this point, we have been configuring the breakfast machine. Now it is time to get serious and actually instruct the machine to boil our eggs, and verify afterwards that we got what we wanted. Add the following piece of code to *eggs.py*:

```
@when(u'machine boils eggs')
def impl(context):
    context.machine.boil_eggs()

@then(u'there should be {amount} boiled egg')
def impl(context, amount):
    assert context.machine.boiled_eggs == int(amount)

@then(u'eggs should be {hardness}')
def impl(context, hardness):
    assert context.machine.boiled_egg_hardness == hardness
```

The first step is to instruct our breakfast machine to boil eggs. The next two are to ascertain that the results of boiling are what we wanted. If you run Behave at this point, an error will be displayed, because the machine does not yet know how to boil eggs. Change *breakfast.py* to the following final version, and the tests should pass:

```
class BreakfastMachine(object):
    def __init__(self):
        super(BreakfastMachine, self).__init__()
        self.eggs = 0
        self.egg_hardness = 'soft'
        self.eggs_to_boil = 0

        self.boiled_eggs = 0
        self.boiled_egg_hardness = None

    def boil_eggs(self):
        if self.eggs_to_boil <= self.eggs:
            self.boiled_eggs = self.eggs_to_boil
            self.boiled_egg_hardness = self.egg_hardness
            self.eggs = self.eggs - self.eggs_to_boil
        else:
            self.boiled_eggs = 0
            self.boiled_egg_hardness = None
```

Now we have a passing scenario and some code; so what's next? You could experiment with what we have now, and see

if you can write another scenario and try to soft-boil an egg, or boil multiple eggs. Just add a new scenario in *boiling\_eggs.feature* after the first one, leaving an empty line in between. Do not repeat the feature or background sections—those are needed only once. After experimenting for a bit, continue to the second specification.

## The second specification

Let's start by reviewing the customer's requirements listed at the beginning of this article.

Boiling varying numbers of eggs to different specifications, i.e., hard or soft, is taken care of. Frying and toasting should be easy to implement, since it is similar to boiling eggs. "Just make easy-to-understand commands and queries that can be used to control the machine," catches our attention, and we ask for a clarification from our customer, who sends us an API specification that tells us how their user interface is going to interact with our machine. Since they are still working on it, we've got only the part that deals with eggs. The following is for *ui.py* in the *breakfast* directory:

```
class BreakfastUI(object):
    def __init__(self):
        super(BreakfastUI, self).__init__()

    def eggs_boiled(self, amount, hardness):
        pass

    def error(self, message):
        pass
```

Their user interface is not ready yet, and their API specification is not completely ready either. But we got some of it, and it is good enough for us to start working with.

We can first tackle sending error messages to the user interface. Let's add the following code at the end of *boiling\_eggs.feature*:

```
Scenario: boiling too many eggs should give an error
    Given machine has 1 eggs
    And amount of eggs to boil is set to 5
    When machine boils eggs
    Then there should be error message "not enough eggs"
```

The next step, like before, is to implement new steps in *eggs.py*:

```
from breakfast import BreakfastMachine
from ui import BreakfastUI
from mockito import verify, mock
from behave import *

@given(u'machine is standing by')
def impl(context):
```